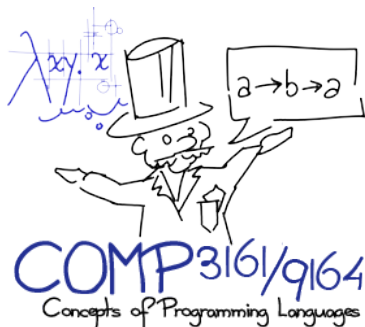




Haskell Concurrency and STM

Johannes Åman Pohjola
UNSW
Term 3 2023

Shared Data

Consider the **Readers and Writers** problem:

Problem

We have a **large shared data structure**, that can't be updated in one atomic step. **Writers** are updating it, and **readers** try to retrieve a coherent copy of it.

We want:

- **Atomicity**: partial updates are not observable.
- **Consistency**: any reader that starts after a finished update will see that update.
- Minimal **waiting**.

A Crappy Solution

Treat both reads and updates as critical sections — use any old critical section solution (locks, etc.) to sequentialise all reads and writes.

A Crappy Solution

Treat both reads and updates as critical sections — use any old critical section solution (locks, etc.) to sequentialise all reads and writes.

Observation

Updates are *atomic* and reads are *consistent*—but reads can't happen concurrently, which leads to unnecessary *contention*.

A Better Solution

A more elaborate locking mechanism (*condition variables*) could be used to to allow multiple concurrent readers. Writers still require exclusive access.

A Better Solution

A more elaborate locking mechanism (*condition variables*) could be used to allow multiple concurrent readers. Writers still require exclusive access.

Observation

This reduces contention. We can't let updates execute concurrently with reads; otherwise, partial updates would be observable.

Reading and Writing

Complication

Suppose we don't want readers to wait _(much) while an update is performed. Instead, we'll give them an *older version* of the data.

Trick: A writer creates *their own local copy* of the data, and then updates just the *pointer* to the data.

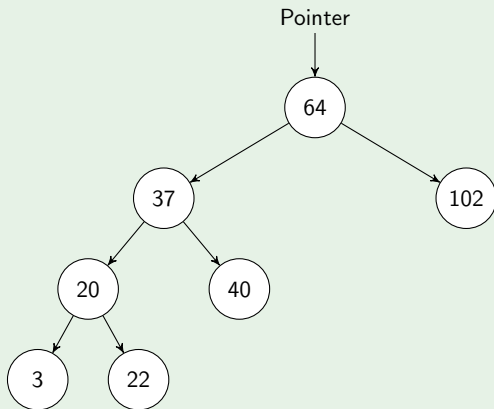
Atomicity The only shared write is now just to one pointer.

Consistency Reads that start before the pointer update get the older version, but reads that start after get the latest.

Persistent Data Structures

Copying is $\mathcal{O}(n)$ in the worst case, but we can do better for many tree-like structures.

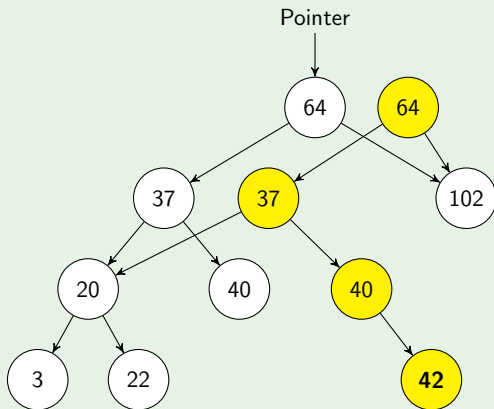
Example (Binary Search Tree)



Persistent Data Structures

Copying is $\mathcal{O}(n)$ in the worst case, but we can do better for many tree-like structures.

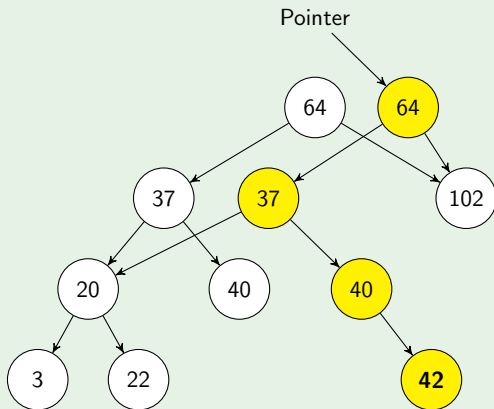
Example (Binary Search Tree)



Persistent Data Structures

Copying is $\mathcal{O}(n)$ in the worst case, but we can do better for many tree-like structures.

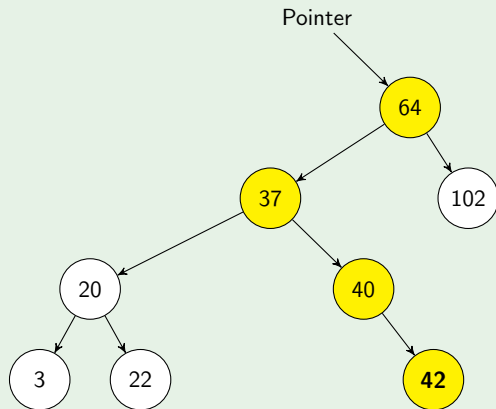
Example (Binary Search Tree)



Persistent Data Structures

Copying is $\mathcal{O}(n)$ in the worst case, but we can do better for many tree-like structures.

Example (Binary Search Tree)



Purely Functional Data Structures

Persistent data structures that use copying (rather than mutation) are called *purely functional* data structures. Operations on them can be expressed as mathematical functions that, given an input structure, return a *new* output structure:

$$\begin{aligned} \text{insert } v \text{ Leaf} &= \text{Branch } v \text{ Leaf Leaf} \\ \text{insert } v (\text{Branch } x \text{ } l \text{ } r) &= \text{if } v \leq x \text{ then} \\ &\quad \text{Branch } x (\text{insert } v \text{ } l) \text{ } r \\ &\quad \text{else} \\ &\quad \text{Branch } x \text{ } l (\text{insert } v \text{ } r) \end{aligned}$$

Computing with Functions

We model processes in Haskell using the **IO** type. We'll treat **IO** as an abstract type for now, and give it a **formal semantics** later if we have time:

$\text{IO } \tau =$ A (possibly effectful) process that, when **executed**, produces a result of type τ

Note the semantics of **evaluation** and **execution** are different things.

Building up IO

Recall **monads**:

$$\text{return} :: \forall a. a \rightarrow \text{IO } a$$
$$(\gg) :: \forall a b. \text{IO } a \rightarrow (a \rightarrow \text{IO } b) \rightarrow \text{IO } b$$
$$\text{getChar} :: \text{IO Char}$$
$$\text{putChar} :: \text{Char} \rightarrow \text{IO } ()$$

Example (Echo)

$$\text{echo} :: \text{IO } ()$$
$$\text{echo} = \text{getChar} \gg (\lambda x. \text{putChar } x \gg \lambda y. \text{echo})$$

Or, with **do** notation:

$$\text{echo} :: \text{IO } ()$$
$$\begin{aligned} \text{echo} = \text{do } & x \leftarrow \text{getChar} \\ & \text{putChar } x \\ & \text{echo} \end{aligned}$$

Adding Concurrency

We can have multiple threads easily enough:

$$\text{forkIO} :: \text{IO } () \rightarrow \text{IO } ()$$

Example (Dueling Printers)

```
let loop c = do putChar c; loop c
in do forkIO (loop 'a'); loop 'z'
```

But what sort of *synchronisation primitives* are available?

MVars

The **MVar** is the simplest synchronisation primitive in Haskell. It can be thought of as a shared box which holds at most one value.

Processes must take the value out of a **full** box to read it, and must put a value into an **empty** box to update it.

MVar Functions

<i>newMVar</i> :: $\forall a. a \rightarrow \text{IO (MVar } a)$	Create a new MVar
<i>takeMVar</i> :: $\forall a. \text{MVar } a \rightarrow \text{IO } a$	Read/remove the value
<i>putMVar</i> :: $\forall a. \text{MVar } a \rightarrow a \rightarrow \text{IO ()}$	Update/insert a value

Taking from an empty MVar or putting into a full one results in blocking.

Readers and Writers

We can treat MVars as shared variables with some definitions:

```
writeMVar m v = do takeMVar m; putMVar m v
readMVar m = do v ← takeMVar m; putMVar m v; return v
```

```
problem :: DB → IO ()
problem initial = do
  db ← newMVar initial
  wl ← newMVar ()
  let reader = readMVar db >>= ...
  let writer = do
    takeMVar wl
    d ← readMVar db
    let d' = update d
    evaluate d'
    writeMVar db d'
    putMVar wl ()
```

Fairness

Each MVar has an attached FIFO queue, so GHC Haskell can ensure the following fairness property:

No thread can be blocked indefinitely on an MVar unless another thread holds that MVar indefinitely.

The Problem with Locks

Problem

Write a procedure to transfer money from one bank account to another. To keep things simple, both accounts are held in memory: no interaction with databases is required. The procedure must operate correctly in a concurrent program, in which many threads may call transfer simultaneously. No thread should be able to observe a state in which the money has left one account, but not arrived in the other (or vice versa).

The Problem with Locks

Assume some infrastructure for accounts:

```
type Balance = Int
```

The Problem with Locks

Assume some infrastructure for accounts:

```
type Balance = Int
```

```
type Account = MVar Balance
```

The Problem with Locks

Assume some infrastructure for accounts:

```
type Balance = Int
```

```
type Account = MVar Balance
```

```
withdraw :: Account → Int → IO ()
```

```
withdraw a m = takeMVar a >>= (putMVar a ∘ subtract m)
```

The Problem with Locks

Assume some infrastructure for accounts:

```
type Balance = Int
```

```
type Account = MVar Balance
```

```
withdraw :: Account → Int → IO ()
```

```
withdraw a m = takeMVar a >>= (putMVar a ∘ subtract m)
```

```
deposit :: Account → Int → IO ()
```

```
deposit a m = withdraw a (−m)
```

Attempt #1

transfer f t $m = \mathbf{do}$ *withdraw* f m ; *deposit* t m

Attempt #1

transfer $f\ t\ m = \mathbf{do}\ \textit{withdraw}\ f\ m; \textit{deposit}\ t\ m$

Problem

The intermediate states where a transaction has only been partially completed are externally observable.

In a bank, we might want the invariant that at all points during the transfer, the total amount of money in the system remains constant. We should have no money go missing.

Attempt #2

```
transfer f t m = do  
    fb ← takeMVar f  
    tb ← takeMVar t  
    putMVar t (tb + m)  
    putMVar f (fb - m)
```

Attempt #2

```
transfer f t m = do  
  fb ← takeMVar f  
  tb ← takeMVar t  
  putMVar t (tb + m)  
  putMVar f (fb - m)
```

Problem

We can have *deadlock* here, when two people transfer to each other simultaneously and both transfers proceed in lock-step.

Also, we can't just compose our existing *withdrawal* and *deposit* operations. That's sad.

Solution

We should enforce a *global* ordering of locks.

type Account = (MVar Balance, AccountNo)

transfer (*f*, *fa*) (*t*, *ta*) *m* = **do**

 (*fb*, *tb*) ← **if** *fa* ≤ *ta*

then do

fb ← *takeMVar f*

tb ← *takeMVar t*

pure (*fb*, *tb*)

else do

tb ← *takeMVar t*

fb ← *takeMVar f*

pure (*fb*, *tb*)

putMVar t (*tb* + *m*)

putMVar f (*fb* − *m*)

It Gets Complicated

Problem

Now suppose that some accounts can be configured with a “backup” account, that is used if insufficient funds are available in the default account.

It Gets Complicated

Problem

Now suppose that some accounts can be configured with a “backup” account, that is used if insufficient funds are available in the default account.

Should you take the lock for the backup account?

It Gets Complicated

Problem

Now suppose that some accounts can be configured with a “backup” account, that is used if insufficient funds are available in the default account.

Should you take the lock for the backup account?

To make life even harder: What if we want to *block* if insufficient funds are available?

Conclusion

Lock-based methods have their place, but from a software engineering perspective they're a nightmare.

Conclusion

Lock-based methods have their place, but from a software engineering perspective they're a nightmare.

- Remember not to take too many locks.

Conclusion

Lock-based methods have their place, but from a software engineering perspective they're a nightmare.

- Remember not to take too many locks.
- Remember not to take too few locks.

Conclusion

Lock-based methods have their place, but from a software engineering perspective they're a nightmare.

- Remember not to take too many locks.
- Remember not to take too few locks.
- Remember what locks correspond to each piece of shared data.

Conclusion

Lock-based methods have their place, but from a software engineering perspective they're a nightmare.

- Remember not to take too many locks.
- Remember not to take too few locks.
- Remember what locks correspond to each piece of shared data.
- Remember not to take the locks in the wrong order.

Conclusion

Lock-based methods have their place, but from a software engineering perspective they're a nightmare.

- Remember not to take too many locks.
- Remember not to take too few locks.
- Remember what locks correspond to each piece of shared data.
- Remember not to take the locks in the wrong order.
- Remember to deal with locks when an error occurs.

Conclusion

Lock-based methods have their place, but from a software engineering perspective they're a nightmare.

- Remember not to take too many locks.
- Remember not to take too few locks.
- Remember what locks correspond to each piece of shared data.
- Remember not to take the locks in the wrong order.
- Remember to deal with locks when an error occurs.
- Remember to signal condition variables and release locks at the right time.

Conclusion

Lock-based methods have their place, but from a software engineering perspective they're a nightmare.

- Remember not to take too many locks.
- Remember not to take too few locks.
- Remember what locks correspond to each piece of shared data.
- Remember not to take the locks in the wrong order.
- Remember to deal with locks when an error occurs.
- Remember to signal condition variables and release locks at the right time.

Most importantly, *modular programming* becomes impossible.

The Solution

Represent an account as a simple shared variable containing the balance.

transfer $f\ t\ m = \text{atomically } \$\ \text{do}$
withdraw $f\ m$
deposit $t\ m$

Where *atomically* P guarantees:

Atomicity The effects of the action P become visible all at once.

Isolation The effects of action P is not affected by any other threads.

Problem

How can we implement *atomically*?

The Global Lock

We can adopt the solution of certain reptilian programming languages.

Problem

Atomicity is guaranteed, but what about *isolation*?

Also, performance is predictably garbage.

Ensuring Isolation

Rather than use regular shared variables, use special *transactional variables*.

$$\begin{aligned} \text{newTVar} &:: a \rightarrow \text{STM} (\text{TVar } a) \\ \text{readTVar} &:: \text{TVar } a \rightarrow \text{STM } a \\ \text{writeTVar} &:: \text{TVar } a \rightarrow a \rightarrow \text{STM} () \\ \\ \text{atomically} &:: \text{STM } a \rightarrow \text{IO } a \end{aligned}$$

The type constructor `STM` is also an instance of the *Monad* type class, and thus supports the same basic operations as `IO`.

$$\begin{aligned} \text{pure} &:: a \rightarrow \text{STM} (\text{TVar } a) \\ (\gg=) &:: \text{STM } a \rightarrow (a \rightarrow \text{STM } b) \rightarrow \text{STM } b \end{aligned}$$

Implementing Accounts

`type Account = TVar Int`

`withdraw :: Account → Int → STM ()`

`withdraw a m = do`

`balance ← readTVar m`

`writeTVar a (balance - m)`

`deposit a m = withdraw a (-m)`

Observe: *withdraw* (resp. *deposit*) can **only** be called inside an *atomically*

Implementing Accounts

`type Account = TVar Int`

`withdraw :: Account → Int → STM ()`

`withdraw a m = do`

`balance ← readTVar m`

`writeTVar a (balance - m)`

`deposit a m = withdraw a (-m)`

Observe: `withdraw` (resp. `deposit`) can **only** be called inside an *atomically* $\Rightarrow$ **We have isolation.**

Implementing Accounts

`type Account = TVar Int`

`withdraw :: Account → Int → STM ()`

`withdraw a m = do`

`balance ← readTVar m`

`writeTVar a (balance - m)`

`deposit a m = withdraw a (-m)`

Observe: *withdraw* (resp. *deposit*) can **only** be called inside an *atomically* ⇒ **We have isolation.**

But, we'd still like to run more than one transaction at once — one global lock isn't good enough.

Optimistic Execution

Each transaction (atomically block) is executed *optimistically*. This means they do not need to check that they are allowed to execute the transaction first (unlike, say, locks, which prefer a *pessimistic* model).

Optimistic Execution

Each transaction (atomically block) is executed *optimistically*. This means they do not need to check that they are allowed to execute the transaction first (unlike, say, locks, which prefer a *pessimistic* model).

Implementation Strategy

Each transaction has an associated *log*, which contains:

- The values written to any *TVars* with *writeTVar*.
- The values read from any *TVars* with *readTVar*, consulting earlier log entries first.

First the log is *validated*, and, if validation succeeds, changes are *committed*. Validation and commit are *one atomic step*.

What can we do if validation fails?

Optimistic Execution

Each transaction (atomically block) is executed *optimistically*. This means they do not need to check that they are allowed to execute the transaction first (unlike, say, locks, which prefer a *pessimistic* model).

Implementation Strategy

Each transaction has an associated *log*, which contains:

- The values written to any *TVars* with *writeTVar*.
- The values read from any *TVars* with *readTVar*, consulting earlier log entries first.

First the log is *validated*, and, if validation succeeds, changes are *committed*. Validation and commit are *one atomic step*.

What can we do if validation fails? **We re-run the transaction!**

Re-running transactions

```
atomically $ do
  x ← readTVar xv
  y ← readTVar yv
  if x > y then detonateTNT else pure ()
```

To avoid harmful side-effects, the transaction must be *repeatable*.
We can't change the world until *commit* time.

A real implementation is smart enough not to retry with exactly the same schedule.

Blocking and *retry*

Problem

We want to *block* if insufficient funds are available.

We can use the helpful action *retry* :: STM a.

Blocking and *retry*

Problem

We want to *block* if insufficient funds are available.

We can use the helpful action *retry* :: STM a.

```
withdraw' :: Account → Int → STM ()  
withdraw' a m = do  
    balance ← readTVar a  
    if m > 0 && m > balance then  
        retry  
    else  
        writeTVar a (balance - m)
```

Choice and *orElse*

Problem

We want to transfer from a backup account if the first account has insufficient funds, and *block* if neither account has insufficient funds.

We can use the helpful action

$$\text{orElse} :: \text{STM } a \rightarrow \text{STM } a \rightarrow \text{STM } a$$

Choice and *orElse*

Problem

We want to transfer from a backup account if the first account has insufficient funds, and *block* if neither account has insufficient funds.

We can use the helpful action

$$\text{orElse} :: \text{STM } a \rightarrow \text{STM } a \rightarrow \text{STM } a$$
$$\text{wdBackup} :: \text{Account} \rightarrow \text{Account} \rightarrow \text{Int} \rightarrow \text{STM } ()$$
$$\text{wdBackup } a_1 \ a_2 \ m = \text{orElse } (\text{withdraw}' \ a_1 \ m) (\text{withdraw}' \ a_2 \ m)$$

Evaluating STM

STM is *modular*. We can compose transactions out of smaller transactions. We can hide concurrency behind library boundaries without worrying about deadlock or global invariants.

Lock-free data structures and transactional memory based solutions work well if contention is low and under those circumstances scale better to higher process numbers than lock-based ones.

Most importantly, the resulting code is often simpler and more robust. **Profit!**

Progress

One transaction can force another to abort only when it commits.

At any time, at least one currently running transaction can successfully commit.

Traditional deadlock scenarios are impossible, as is cyclic restarting where two transactions constantly cancel each other.

Starvation is possible (*when?*), however uncommon in practice. So, we technically don't have *eventual entry*.

Database Guarantees

Atomicity ✓ Each transaction should be ‘all or nothing’.

Consistency ✓ Each transaction in the future sees the effects of transactions in the past.

Isolation ✓ The transaction’s effect on the state cannot be affected by other transactions.

Durability The transaction’s effect on the state survives power outages and crashes.

STM gives you 75% of a database system. The Haskell package *acid-state* builds on STM to give you all four.

That's it

We have now covered all the content in COMP3161/COMP9164.
Thanks for sticking with the course.

- **Syntax Foundations**

Concrete/Abstract Syntax, Ambiguity, HOAS, Binding, Variables, Substitution, λ -calculus

- **Semantics Foundations**

Static Semantics, Dynamic Semantics (Small-Step/Big-Step), Abstract Machines, Environments, Stacks, Safety, Liveness, Type Safety (Progress and Preservation)

- **Features**

- Algebraic Data Types, Recursive Types
- Exceptions
- Polymorphism, Type Inference, Unification
- Overloading, Subtyping, Abstract Data Types
- Concurrency, Critical Sections, STM

MyExperience

Please fill out the survey. It helps tremendously.

<https://myexperience.unsw.edu.au>

Further Learning

- UNSW courses:
 - COMP3141 — Software System Design and Implementation
 - COMP6721 — (In-)formal Methods
 - COMP3131 — Compilers
 - COMP4141 — Theory of Computation
 - COMP3151 — Foundations of Concurrency
 - COMP4161 — Advanced Topics in Verification
 - COMP3153 — Algorithmic Verification
- Online Learning
 - Oregon Programming Languages Summer School Lectures (<https://www.cs.uoregon.edu/research/summerschool/archives.html>) Videos are available from here! Also some on YouTube.

What's next?

The exam is on **Wednesday, 6th of December 2022** in the morning session (8-12).

- I have posted a sample exam with revision questions.
- The final exam will run similar to the sample exam.
- It runs for 2 hours and 10 minutes.

Evaluation Semantics

The semantics of Haskell's evaluation are interesting but not particularly relevant for us. We will assume that it happens quietly without a fuss:

$$\beta\text{-equivalence} \quad (\lambda x. M[x]) N \equiv_{\beta} M[N]$$

$$\alpha\text{-equivalence} \quad \lambda x. M[x] \equiv_{\alpha} \lambda y. M[y]$$

$$\eta\text{-equivalence} \quad \lambda x. M x \equiv_{\eta} M$$

Let our ambient congruence relation $\equiv$ be $\equiv_{\alpha\beta\eta}$ enriched with the following extra equations, justified by the *monad laws*:

$$\text{return } N \gg= M \equiv M N$$

$$(X \gg= Y) \gg= Z \equiv X \gg= (\lambda x. Y x \gg= Z)$$

$$X \equiv X \gg= \text{return}$$

Processes

This means that a Haskell expression of type $\text{IO } \tau$ will boil down to either *return* x where x is a value of type τ ; or $a \gg= M$ where a is some *primitive IO action* (*forkIO* p , *readMVar* v , etc.) and M is some function producing another $\text{IO } \tau$. This is the *head normal form* for IO expressions.

Definition

Define a language of *processes* P , which contains all (head-normal) expressions of type $\text{IO } ()$.

We want to define the semantics of the *execution* of these processes. Let's use *operational semantics*:

$$(\mapsto) \subseteq P \times P$$

Semantics for forkIO

To model *forkIO*, we need to model the parallel execution of multiple processes in our process language. We shall add a *parallel composition* operator to the language of processes:

$$\begin{array}{lcl}
 P, Q & ::= & a \gg M \\
 & | & \text{return } () \\
 & | & P \parallel Q \\
 & | & \dots
 \end{array}$$

Semantics for forkIO

To model *forkIO*, we need to model the parallel execution of multiple processes in our process language. We shall add a *parallel composition* operator to the language of processes:

$$\begin{array}{lcl} P, Q & ::= & a \gg M \\ & | & \text{return } () \\ & | & P \parallel Q \\ & | & \dots \end{array}$$

And the following ambient congruence equations:

$$\begin{array}{lcl} P \parallel Q & \equiv & Q \parallel P \\ P \parallel (Q \parallel R) & \equiv & (P \parallel Q) \parallel R \end{array}$$

Semantics for forkIO

If we have multiple processes active, pick one of them non-deterministically to move:

$$\frac{P \mapsto P'}{P \parallel Q \mapsto P' \parallel Q}$$

Semantics for forkIO

If we have multiple processes active, pick one of them non-deterministically to move:

$$\frac{P \mapsto P'}{P \parallel Q \mapsto P' \parallel Q}$$

The *forkIO* operation introduces a new process:

$$(forkIO\ P \gg\!\!= M) \mapsto P \parallel (return\ () \gg\!\!= M)$$

Semantics for MVars

MVars are modelled as a special type of *process*, identified by a *unique name*. Values of `MVar` type merely contain the name of the process, so that `putMVar` and friends know where to look.

$$\begin{array}{lcl}
 P, Q & ::= & a \gg= M \\
 & | & \text{return } () \\
 & | & P \parallel Q \\
 & | & \langle n \rangle \quad | \quad \langle v \rangle_n \\
 & | & \dots
 \end{array}$$

Semantics for MVars

MVars are modelled as a special type of *process*, identified by a *unique name*. Values of `MVar` type merely contain the name of the process, so that `putMVar` and friends know where to look.

$$\begin{array}{lcl}
 P, Q & ::= & a \gg= M \\
 & | & \text{return } () \\
 & | & P \parallel Q \\
 & | & \langle n \rangle \quad | \quad \langle v \rangle_n \\
 & | & \dots
 \end{array}$$

$$\langle n \rangle \parallel (\text{putMVar } n \ v \gg= M) \quad \mapsto \quad \langle v \rangle_n \parallel (\text{return } () \gg= M)$$

Semantics for MVars

MVars are modelled as a special type of *process*, identified by a *unique name*. Values of `MVar` type merely contain the name of the process, so that *putMVar* and friends know where to look.

$$\begin{array}{lcl}
 P, Q & ::= & a \gg= M \\
 & | & \text{return } () \\
 & | & P \parallel Q \\
 & | & \langle \rangle_n \mid \langle v \rangle_n \\
 & | & \dots
 \end{array}$$

$$\langle \rangle_n \parallel (\text{putMVar } n \ v \gg= M) \rightarrow \langle v \rangle_n \parallel (\text{return } () \gg= M)$$

$$\langle v \rangle_n \parallel (\text{takeMVar } n \gg= M) \rightarrow \langle \rangle_n \parallel (\text{return } v \gg= M)$$

Semantics for newMVar

We might think that *newMVar* should have semantics like this:

$$\frac{}{(newMVar\ v \gg\!\!=\ M) \mapsto \langle v \rangle_n \parallel (return\ n \gg\!\!=\ M)} \quad (n\ \text{fresh})$$

But this approach has a number of problems:

- The name *n* is now globally-scoped, without an explicit binder to introduce it.
- It doesn't accurately model the *lifetime* of the MVar, which should be garbage-collected once all processes that can access it finish.
- It makes MVars *global* objects, so our semantics aren't very *abstract*. We would like local communication to be local in our model.

Restriction Operator

We introduce a *restriction operator* ν to our language of processes:

$$\begin{array}{lcl} P, Q & ::= & a \gg= M \\ & | & \text{return } () \\ & | & P \parallel Q \\ & | & \langle \rangle_n \mid \langle v \rangle_n \\ & | & (\nu n) P \end{array}$$

Restriction Operator

We introduce a *restriction operator* ν to our language of processes:

$$\begin{array}{lcl}
 P, Q & ::= & a \gg= M \\
 & | & \text{return } () \\
 & | & P \parallel Q \\
 & | & \langle \rangle_n \mid \langle v \rangle_n \\
 & | & (\nu n) P
 \end{array}$$

Writing $(\nu n) P$ says that the MVar name n is *only* available in process P . Mentioning n outside P is not well-formed. We need the following additional congruence equations:

$$\begin{array}{lcl}
 (\nu n) (\nu m) P & \equiv & (\nu m) (\nu n) P \\
 (\nu n) (P \parallel Q) & \equiv & P \parallel (\nu n) Q \quad (\text{if } n \notin P)
 \end{array}$$

Better Semantics for newMVar

The rule for *newMVar* is much the same as before, but now we explicitly restrict the MVar to *M*.

$$\frac{}{(newMVar\ v \gg\!\!=\ M) \mapsto (\nu\ n)(\langle v \rangle_n \parallel (return\ n \gg\!\!=\ M))}^{(n\ \text{fresh})}$$

Better Semantics for newMVar

The rule for *newMVar* is much the same as before, but now we explicitly restrict the MVar to *M*.

$$\frac{}{(newMVar\ v \gg\!\!= M) \mapsto (\nu\ n)(\langle v \rangle_n \parallel (return\ n \gg\!\!= M))}^{(n\ \text{fresh})}$$

We can always execute under a restriction:

$$\frac{P \mapsto P'}{(\nu\ n)\ P \mapsto (\nu\ n)\ P'}$$

Better Semantics for newMVar

The rule for *newMVar* is much the same as before, but now we explicitly restrict the MVar to *M*.

$$\frac{}{(newMVar\ v \gg\!\!=\ M) \mapsto (\nu\ n)(\langle v \rangle_n \parallel (return\ n \gg\!\!=\ M))}^{(n\ \text{fresh})}$$

We can always execute under a restriction:

$$\frac{P \mapsto P'}{(\nu\ n)\ P \mapsto (\nu\ n)\ P'}$$

Question

What happens when you put an MVar inside another MVar?

Garbage Collection

If an MVar is no longer used, we just replace it with the do-nothing process:

$$\begin{aligned}(\nu n) \langle \rangle_n &\mapsto \text{return } () \\(\nu n) \langle v \rangle_n &\mapsto \text{return } ()\end{aligned}$$

Garbage Collection

If an MVar is no longer used, we just replace it with the do-nothing process:

$$\begin{aligned}(\nu n) \langle \rangle_n &\mapsto \text{return } () \\(\nu n) \langle v \rangle_n &\mapsto \text{return } ()\end{aligned}$$

Extra processes that have outlived their usefulness disappear:

$$\text{return } () \parallel P \mapsto P$$

Process Algebra

Our language P is called a *process algebra*, a common means of describing semantics for concurrent programs.

Bibliography



Simon Marlow

Parallel and Concurrent Programming in Haskell

O'Reilly, 2013

<http://chimera.labs.oreilly.com/books/1230000000929>



Simon Peyton Jones, Andrew Gordon and Sigbjørn Finne

Concurrent Haskell

POPL '96

Association for Computer Machinery

<http://microsoft.com/en-us/research/wp-content/uploads/1996/01/concurrent-haskell.pdf>



Simon Marlow (Editor)

Haskell 2010 Language Report

<https://www.haskell.org/onlinereport/haskell2010/>

Bibliography



Simon Peyton Jones

Beautiful Concurrency

In "Beautiful Code", ed. Greg Wilson, O'Reilly, 2007

<http://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/beautiful.pdf>



Tim Harris, Simon Marlow, Simon Peyton Jones, Maurice Herlihy

Composable Memory Transactions

PPoP '05

Association for Computer Machinery

www.microsoft.com/en-us/research/wp-content/uploads/2005/01/2005-ppopp-composable.pdf



David Himmelstrup

Acid-State Library

<https://github.com/acid-state/acid-state>



Ryan Yates and Michael L. Scott

A Hybrid TM for Haskell

TRANSACTION '14

Association for Computer Machinery

<http://transact2014.cse.lehigh.edu/yates.pdf>